



# Reporte Técnico de Actividades Práctico-Experimentales Nro. 1

## 1. Datos de Identificación del Estudiante y la Práctica

|                                          |                                                                                                                                          |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Nombre del estudiante(s)                 | Melissa Contento, Jaime Landazuri,<br>Alejandro Padilla Espinoza, Domenica<br>Rojas, Ivett Zaragocin                                     |
| Asignatura                               | Sistemas Digitales                                                                                                                       |
| Ciclo                                    | 5                                                                                                                                        |
| Unidad                                   | 1 – Microcontroladores para sistemas<br>embebidos                                                                                        |
| Resultado de aprendizaje de la<br>unidad | R1. Diseña arquitectura de<br>microcontroladores, bajo los principios<br>de solidaridad, transparencia,<br>responsabilidad y honestidad. |
| Práctica Nro.                            | 1                                                                                                                                        |
| Tipo                                     | Grupal                                                                                                                                   |
| Título de la Práctica                    | Uso de Wokwi y Microchip Studio en<br>microcontroladores AVR: arquitectura,<br>puertos de E/S y operaciones a nivel de<br>bit            |
| Nombre del Docente                       | Ing. Luis Darío Sinche C.                                                                                                                |
| Fecha                                    | 2026-10-03                                                                                                                               |
| Horario                                  | Jueves 12h30 a 13h30; Viernes 11h30 a<br>13h30                                                                                           |
| Lugar                                    | Laboratorio de computación (práctica<br>100 % simulada)                                                                                  |
| Tiempo planificado en el Sílabo          | 3 horas                                                                                                                                  |



## 2. Objetivo(s) de la Práctica

**Objetivo general.** Controlar las entradas y salidas digitales del microcontrolador ATmega328P escribiendo y leyendo directamente sus registros mediante operaciones a nivel de bit, con simulación en Wokwi y Compiler Explorer.

Objetivos específicos:

- Relacionar los pines de la placa Arduino UNO con los puertos y bits del ATmega328P (PB, PC, PD).
- Configurar entradas y salidas digitales con los registros DDRx, PORTx y PINx usando máscaras.
- Comparar el tiempo de ejecución de la API de Arduino con el del acceso directo a registros.
- Identificar en el ensamblador generado las instrucciones de manejo de bits del set AVR.

## 3. Fundamento Teórico

Un microprocesador es solo la CPU y necesita memoria y periféricos externos; un microcontrolador los integra en un chip (Freire Sánchez, 2023; Valdivieso y Solís, 2019).

Tabla 1. Características del ATmega328P (Microchip Technology Inc., 2024).

| Característica           | Valor                        |
|--------------------------|------------------------------|
| Arquitectura             | AVR de 8 bits, RISC, Harvard |
| Memoria Flash (programa) | 32 KB                        |
| SRAM (datos)             | 2 KB                         |
| EEPROM                   | 1 KB                         |
| Registros de trabajo     | R0–R31 (32)                  |
| Frecuencia de reloj      | 16 MHz                       |

El ATmega328P es Harvard: programa y datos tienen buses y espacios de direcciones separados, y la Flash se lee con LPM.

La mayoría de las instrucciones dura un ciclo de reloj:

$$T = \frac{1}{f} = \frac{1}{16 \text{ MHz}} = 62,5 \text{ ns}$$

Cada pin del UNO pertenece a un puerto del ATmega328P.

Tabla 2. Correspondencia de pines del Arduino UNO.

| Pines UNO | Puerto | Bits                          |
|-----------|--------|-------------------------------|
| D0–D7     | PORTD  | PD0–PD7                       |
| D8–D13    | PORTB  | PB0–PB5 (LED “L” = D13 = PB5) |
| A0–A5     | PORTC  | PC0–PC5                       |

Cada pin se controla con un bit en tres registros.



Tabla 3. Registros de un puerto de E/S del ATmega328P.

| Registro | Función                   | Bit en 0           | Bit en 1              |
|----------|---------------------------|--------------------|-----------------------|
| DDRx     | Dirección del pin         | Entrada            | Salida                |
| PORTx    | Nivel de salida o pull-up | Bajo / sin pull-up | Alto / pull-up activa |
| PINx     | Leer el pin               | 0 V                | 5 V                   |

Escribir un 1 en PINx invierte el bit de PORTx.

Para cambiar solo el bit  $n$  se usa una máscara, que en C se escribe  $1 \ll n$ :

$$m = 2^n$$

Tabla 4. Patrones de manipulación de bits.

| Operación             | Expresión en C                                    | Idea                           |
|-----------------------|---------------------------------------------------|--------------------------------|
| Poner en 1 el bit $n$ | $\text{REG}  = (1 \ll n);$                        | $R \leftarrow R \vee m$        |
| Poner en 0 el bit $n$ | $\text{REG} \&= \sim(1 \ll n);$                   | $R \leftarrow R \wedge \neg m$ |
| Invertir el bit $n$   | $\text{REG} \hat{=} (1 \ll n);$                   | $R \leftarrow R \oplus m$      |
| Leer el bit $n$       | $(\text{REG} \gg n) \& 1$                         | $(R \gg n) \wedge 1$           |
| Escribir un campo     | $\text{REG} = (\text{REG} \& \sim M)   (v \& M);$ | borrar el campo y poner $v$    |

Tabla 5. Instrucciones AVR usadas (Microchip Technology Inc., s.f.).

| Instrucción | Operación                                | Ciclos          |
|-------------|------------------------------------------|-----------------|
| SBI / CBI   | Poner / borrar un bit de E/S (0x00–0x1F) | 2               |
| SBIS        | Saltar si un bit de E/S es 1             | 1 (2 con salto) |
| IN / OUT    | Leer / escribir un registro de E/S       | 1               |
| ORI         | OR con una constante                     | 1               |
| EOR         | XOR entre registros                      | 1               |
| LDI         | Cargar una constante en un registro      | 1               |
| RJMP        | Salto relativo                           | 2               |

En el listado aparecen direcciones de E/S (PORTB = 0x05) y no de datos (0x25): SBI, CBI, IN y OUT usan el espacio de E/S.

`digitalWrite()` es portable pero más lenta que el acceso directo (Arduino, 2024); Wokwi permite simular todo sin hardware (CodeMagic LTD., 2025).

## 4. Materiales, Reactivos, Equipos y Herramientas

**Componentes virtuales (Wokwi):** 1 Arduino UNO (ATmega328P a 16 MHz), 4 LEDs, 4 resistencias de 220  $\Omega$  y 1 pulsador; sin componentes físicos.

**Herramientas:** navegador, Wokwi, Compiler Explorer (AVR gcc 5.4.0, -O1) en lugar de Microchip Studio, hoja de datos del ATmega328P y AVR Instruction Set Manual. Códigos y enlaces: <https://github.com/AlejandroTatum/sistemas-digitales-ape1-a-vr> (ver Anexos).

## 5. Procedimiento / Metodología Ejecutada

Desde la Parte B no se usan `pinMode()`, `digitalWrite()` ni `digitalRead()`.

### 5.1. Parte A. Reconocimiento del entorno

**Datos y montaje.** Arduino UNO en Wokwi con el Código 1 de la guía (delay(500) entre HIGH y LOW en D13). Código: parte-a/.

1. Se creó el proyecto APE1\_Contento\_Landazuri\_Rojas\_Padilla\_Zaragocin y se ubicaron el ATmega328P, el LED “L” y el pin D13.
2. Se ejecutó el Código 1 (Figuras 1 y 2).

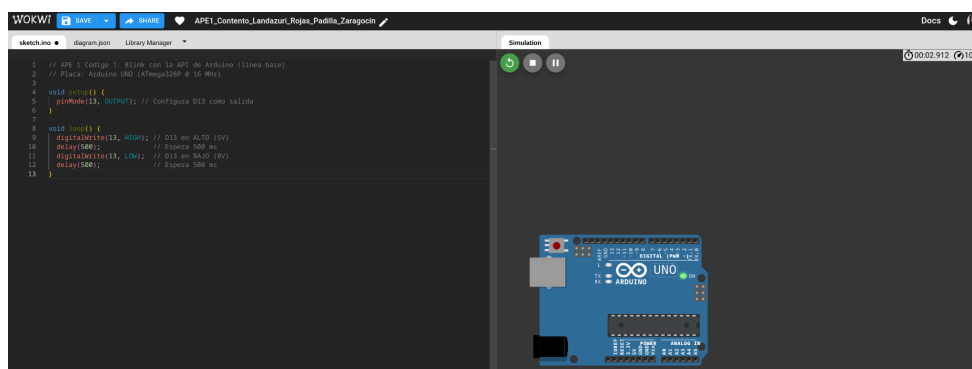


Figura 1: Código 1 en Wokwi con el LED “L” apagado

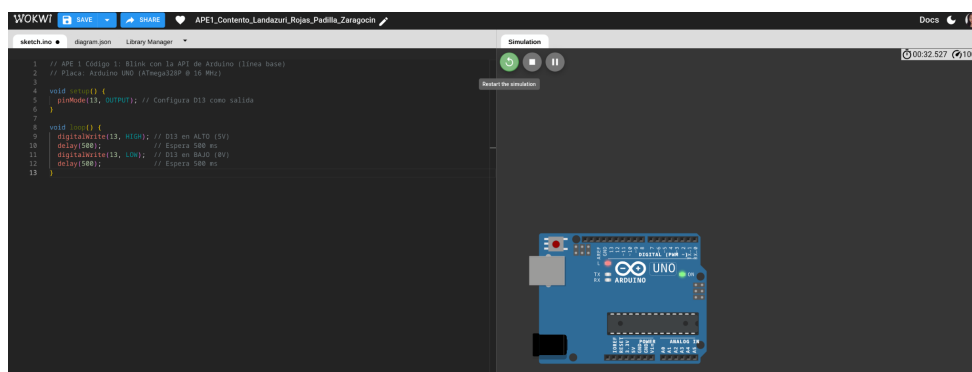


Figura 2: Código 1 en Wokwi con el LED “L” encendido

**Resultado y justificación.** D13 es el bit 5 del puerto B (**PB5**), por lo que el LED se controla con la máscara  $1 \ll 5$  sobre DDRB, PORTB y PINB. Con 500 ms encendido y 500 ms apagado,  $T = 1$  s y  $f = 1$  Hz, como indica la guía.

## 5.2. Parte B. El mismo comportamiento con registros

**Datos y montaje.** Mismo UNO. Wokwi (versión B2): <https://wokwi.com/projects/476790602516086785>. Código: parte-b/.

```
1 PORTB ^= (1 << PORTB5); // B1, Código 2: XOR conmuta el bit 5
2 PORTB |= (1 << PORTB5); // B2: encender (OR) ...
3 PORTB &= ~(1 << PORTB5); // B2: ... y apagar (AND con la máscara invertida)
4 PINB = (1 << PINB5); // B3: escribir 1 en PINB conmuta PB5
5 PORTB = (1 << PORTB5); // B4: asignación del byte completo (enciende) ...
6 PORTB = 0; // B4: ... y asignación del byte en 0 (apaga)
```

1. **B1.** Código 2 con XOR (Figura 3). **B2.**  $|=$  y  $\&= \sim$  (Figura 4).
2. **B3.**  $PINB = (1 \ll PINB5)$ ; (Figura 5). **B4.**  $PORTB = \dots$  y  $PORTB = 0$ ; (Figura 6).

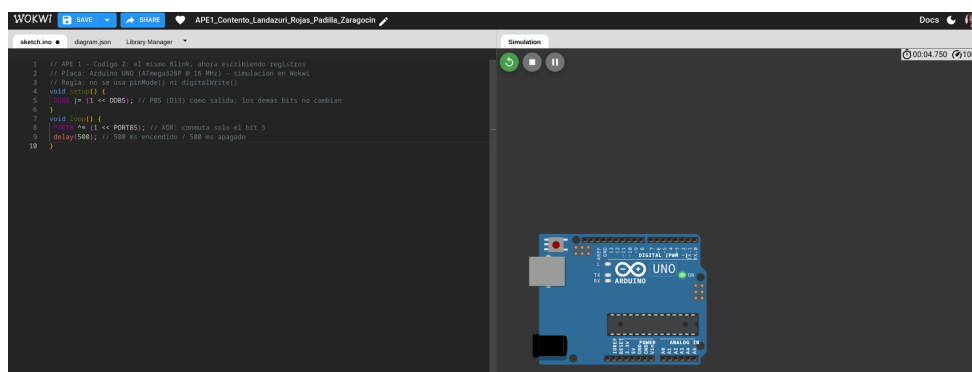


Figura 3: B1: Código 2 con XOR

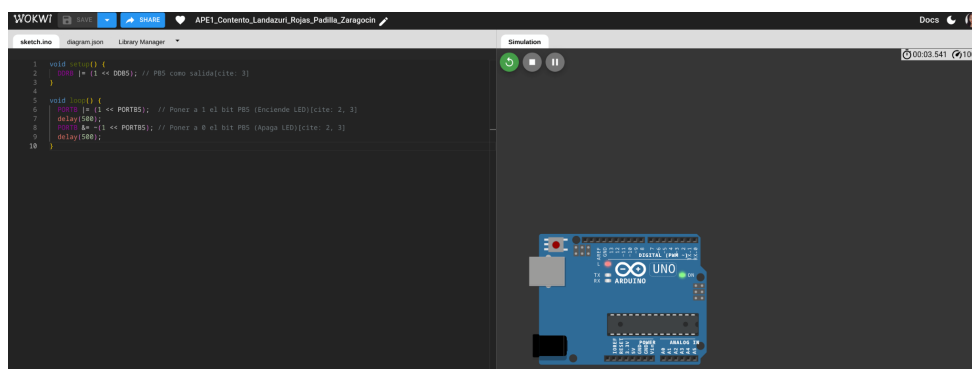
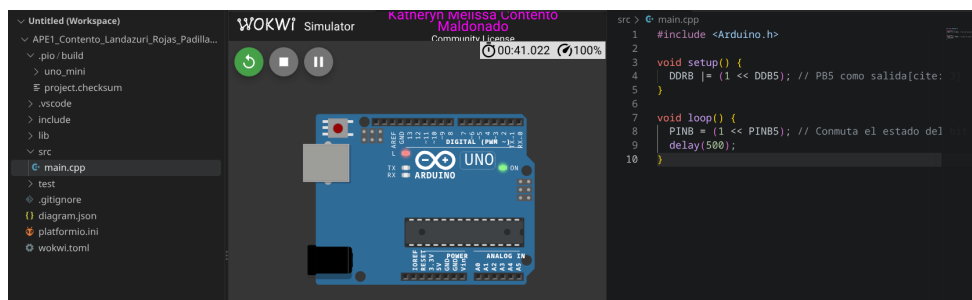
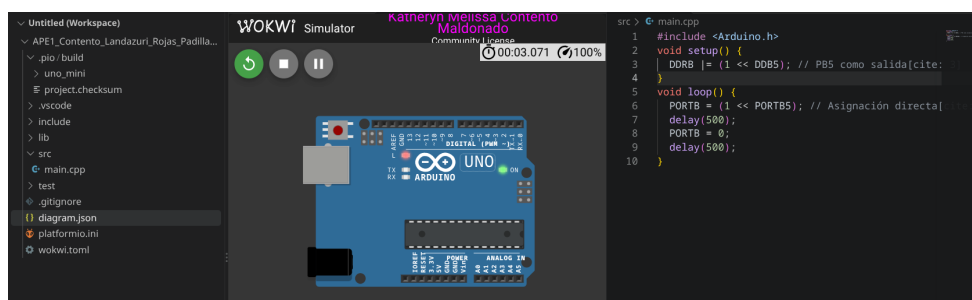


Figura 4: B2: encendido con OR y apagado con AND



**Figura 5:** B3: conmutación escribiendo en PINB



**Figura 6:** B4: asignación directa a PORTB

**Resultado y justificación.** Los cuatro pasos parpadean cada 500 ms. XOR, OR y AND con máscara cambian solo el bit 5. En B3, escribir un 1 en PINx invierte por hardware el bit de PORTx, sin secuencia leer-modificar-escribir (Microchip Technology Inc., 2024). En B4, PORTB toma 00100000 y pone en 0 los otros siete bits; no se nota porque PB5 es la única salida (se comprueba en C6).

### 5.3. Parte C. Circuito y laboratorio de bits

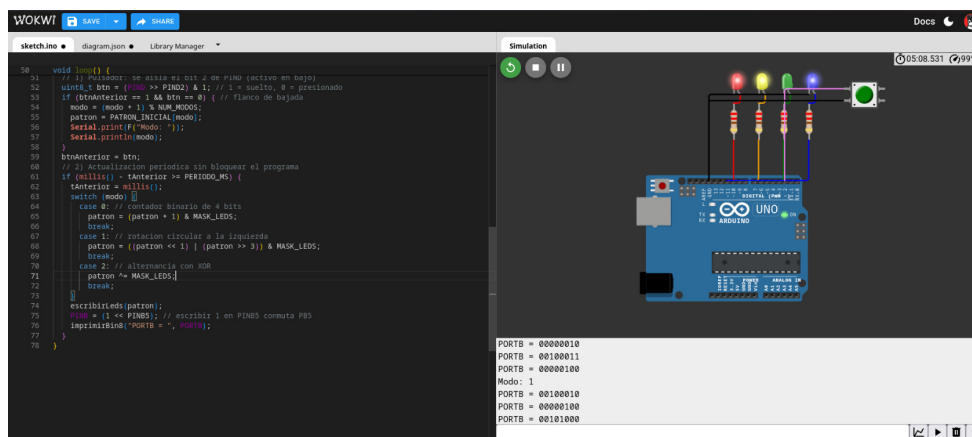
**Datos y montaje.** Circuito de la Tabla 6 con el diagram.json de la guía. Wokwi: <https://wokwi.com/projects/476794088291938305>. Código 3: parte-c/.

Tabla 6. Conexiones pin a pin del circuito de la Parte C.

| Componente                        | Pin del componente | Pin Arduino UNO | Puerto / bit | Configuración               |
|-----------------------------------|--------------------|-----------------|--------------|-----------------------------|
| LED0 (rojo) + R0 220 $\Omega$     | Ánodo vía R0       | D8              | PB0          | Salida                      |
| LED1 (amarillo) + R1 220 $\Omega$ | Ánodo vía R1       | D9              | PB1          | Salida                      |
| LED2 (verde) + R2 220 $\Omega$    | Ánodo vía R2       | D10             | PB2          | Salida                      |
| LED3 (azul) + R3 220 $\Omega$     | Ánodo vía R3       | D11             | PB3          | Salida                      |
| LED0 ... LED3                     | Cátodo             | GND             | —            | —                           |
| Pulsador                          | Terminal 1 / 2     | D2 / GND        | PD2          | Entrada con pull-up interna |
| LED integrado "L"                 | —                  | D13             | PB5          | Salida                      |

- C1.** Se armó el circuito de la Tabla 6 (Figura 7).
- C2.** Se cargó el Código 3 y se abrió el Monitor Serie a 9600 baudios.

3. **C3.** Se calcularon a mano los resultados de `demoOperaciones()`.
4. **C4.** Se recorrieron los tres modos de la Tabla 7 con el pulsador.
5. **C5.** Se leyó `PORTB = ...` en el Monitor Serie (Tabla 8).
6. **C6.** Se cambió `escribirLeds()` por `PORTB = valor;` y luego se restauró.



**Figura 7:** Parte C en Wokwi: LEDs en PB0–PB3, pulsador en D2 y Monitor Serie con líneas `PORTB = ...` y `Modo: 1`

Cálculo de C3 con  $a = 11001100_2$  y  $b = 10101010_2$  (coincide con el Monitor Serie):

```

1 a      = 1 1 0 0 1 1 0 0
2 b      = 1 0 1 0 1 0 1 0
3 a & b  = 1 0 0 0 1 0 0 0  -> 10001000
4 a | b  = 1 1 1 0 1 1 1 0  -> 11101110
5 a ^ b  = 0 1 1 0 0 1 1 0  -> 01100110
6 ~a     = 0 0 1 1 0 0 1 1  -> 00110011 (con (uint8_t), 8 bits)
7 a << 1 = 10011000
8 a >> 2 = 00110011

```

Tabla 7. Modos de C4 (patrón de 4 bits, actualizado cada 300 ms).

| Modo           | Expresión                                   | Efecto                                                |
|----------------|---------------------------------------------|-------------------------------------------------------|
| 0, contador    | $(patron + 1) \& 0x0F$                      | 0x0F descarta el desbordamiento: 1111 + 1 pasa a 0000 |
| 1, rotación    | $((patron \ll 1)   (patron \gg 3)) \& 0x0F$ | Gira el nibble: 0001, 0010, 0100, 1000, 0001          |
| 2, alternancia | $patron \hat{=} 0x0F$                       | Invierte solo el nibble de los LEDs: 0101 y 1010      |

Tabla 8. Decodificación de las líneas PORTB = ... del Monitor Serie (Figura 7).

| Línea    | LED "L" (bit 5) | LEDs (bits 3-0) | Lectura                          |
|----------|-----------------|-----------------|----------------------------------|
| 00000010 | 0               | 0010            | modo 0, contador                 |
| 00100011 | 1               | 0011            | modo 0, contador                 |
| 00000100 | 0               | 0100            | modo 0, contador                 |
| Modo: 1  | —               | —               | se pulsó el botón                |
| 00100010 | 1               | 0010            | modo 1, rotación<br>(desde 0001) |
| 00000100 | 0               | 0100            | modo 1, rotación                 |
| 00101000 | 1               | 1000            | modo 1, rotación                 |

Tabla 9. Configuración de registros (Tabla 4 de la guía).

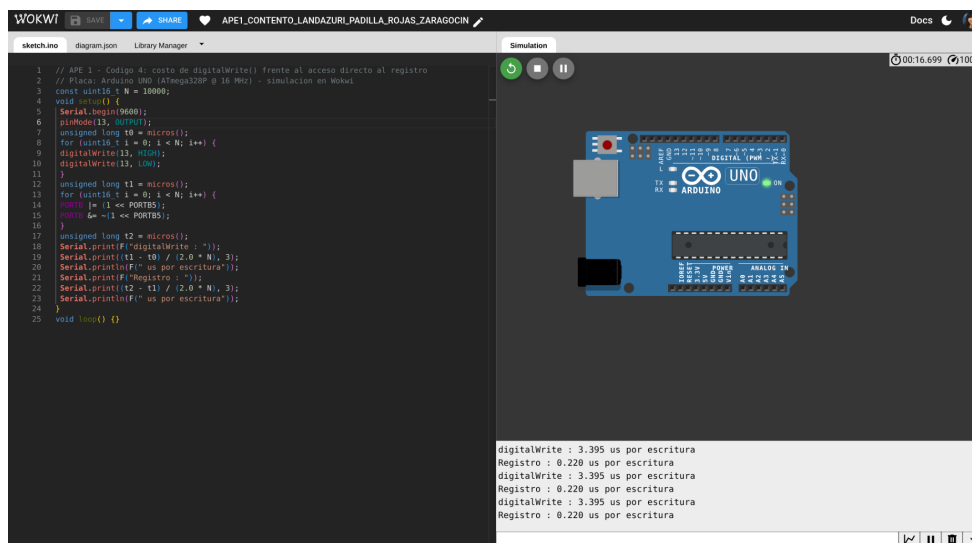
| Pin       | Función             | Bit DDRx | Bit PORTx | Bit PINx leído |
|-----------|---------------------|----------|-----------|----------------|
| D8 (PB0)  | LED0 encendido      | 1        | 1         | —              |
| D11 (PB3) | LED3 apagado        | 1        | 0         | —              |
| D2 (PD2)  | Pulsador suelto     | 0        | 1         | 1              |
| D2 (PD2)  | Pulsador presionado | 0        | 1         | 0              |
| D13 (PB5) | LED "L" encendido   | 1        | 1         | —              |

**Resultado y justificación.** Con  $V_f \approx 2 \text{ V}$  (supuesto),  $I \approx 3 \text{ V} / 220 \Omega \approx 13,6 \text{ mA}$  por LED, bajo el máximo absoluto de 40 mA por pin (se recomiendan unos 20 mA) (Microchip Technology Inc., 2024). En la Tabla 9, con DDR = 0 y PORT = 1 la pull-up lleva D2 a 5 V (PIN = 1) y al presionar el GND la vence (PIN = 0). En C6, PORTB = valor; pone en 0 el bit 5 y la línea siguiente lo conmuta a 1, así que el LED "L" queda encendido: el problema de B4 con más salidas.

#### 5.4. Parte D. Costo de la API frente al acceso a registros

**Datos y montaje.** Código 4 de la guía, ejecutado tres veces; cada bucle hace  $N = 10000$  iteraciones con dos escrituras, así que el tiempo por escritura es  $\Delta t / (2N)$ . Código: parte-d/.





**Figura 8:** Código 4 y resultados de las tres ejecuciones en el Monitor Serie

Tabla 10. Mediciones de la Parte D.

| Ejecución | digitalWrite (µs por escritura) | Registro (µs por escritura) |
|-----------|---------------------------------|-----------------------------|
| 1         | 3,395                           | 0,220                       |
| 2         | 3,395                           | 0,220                       |
| 3         | 3,395                           | 0,220                       |
| Promedio  | 3,395                           | 0,220                       |

$$\text{Relación digitalWrite/registro} = \frac{3,395}{0,220} \approx 15$$

$$\frac{3,395 \mu s}{0,0625 \mu s} \approx 54 \text{ ciclos} \quad \frac{0,220 \mu s}{0,0625 \mu s} \approx 3,5 \text{ ciclos}$$

**Resultado y justificación.** digitalWrite() busca timer, puerto y máscara en tablas de Flash, revisa el PWM y guarda y restaura SREG con cli (según wiring\_digital.c); por eso cuesta  $\approx 54$  ciclos. Los  $0,220 \mu s$  ( $\approx 3,5$  ciclos) quedan por debajo del mínimo esperado: sbi, cbi, el contador de 16 bits y el salto suman  $\geq 8$  ciclos por iteración,  $\approx 4$  por escritura. Como ningún efecto del hardware puede bajar de ese mínimo, el valor refleja que el modelo de tiempo de Wokwi no es exacto al ciclo; por eso no es el costo real de una escritura y la relación  $\approx 15$  es solo un orden de magnitud (Figura 8). Las tres ejecuciones son idénticas porque la simulación es determinista.

## 5.5. Parte E. Del C al set de instrucciones

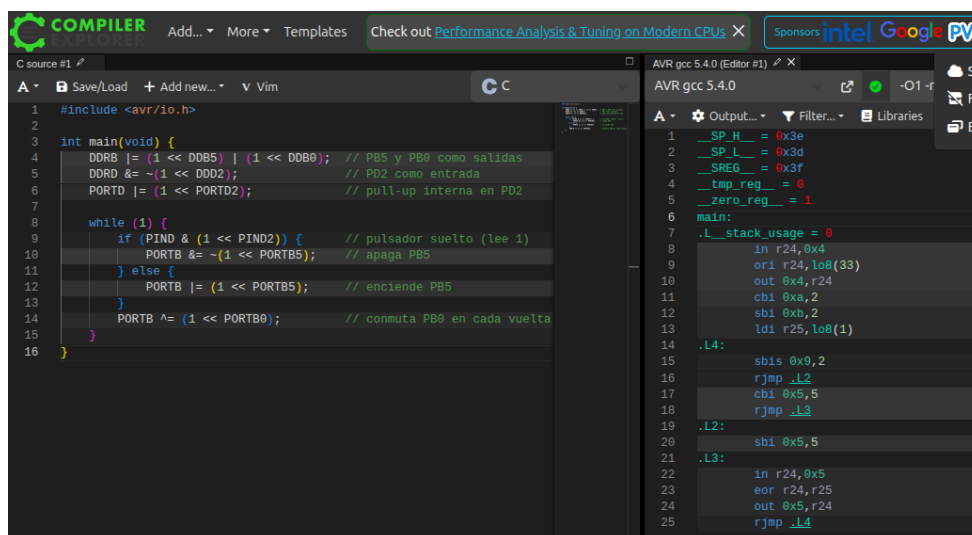
**Datos y montaje.** Compiler Explorer con AVR gcc 5.4.0 y -O1 -mmcu=atmega328p (alternativa permitida por la guía). Enlace: <https://godbolt.org/z/MYTMrn81l>. Código: parte-e/.

1. Se pegó el Código 5 y se leyó el ensamblador (Figura 9).
2. Se completó la Tabla 11 y se contaron los ciclos de una vuelta del while(1).

```

1      in   r24,0x4      ; DDRB
2      ori  r24,33      ; 33 = 0x21 = bits 5 y 0
3      out  0x4,r24
4      cbi  0xa,2        ; DDRD2 = 0
5      sbi  0xb,2        ; PORTD2 = 1 (pull-up)
6      ldi  r25,1        ; máscara de PB0, fuera del lazo
7 .L4:  sbis  0x9,2        ; ¿PIND2 = 1 (suelto)?
8      rjmp .L2
9      cbi  0x5,5        ; suelto: apaga PB5
10     rjmp .L3
11 .L2:  sbi  0x5,5        ; presionado: enciende PB5
12 .L3:  in   r24,0x5
13     eor  r24,r25      ; PB0 ^= 1
14     out  0x5,r24
15     rjmp .L4

```



**Figura 9:** Código C y ensamblador generado en Compiler Explorer

Tabla 11. Correspondencia C – ensamblador (AVR gcc 5.4.0, -O1) (Tabla 5 de la guía).

| Línea en C                         | Instrucción(es) AVR                                                 | Ciclos                          |
|------------------------------------|---------------------------------------------------------------------|---------------------------------|
| DDRB  = (1 << DDB5)   (1 << DDB0); | in r24,0x4; ori r24,33; out 0x4,r24                                 | 3                               |
| DDRD &= ~(1 << DDD2);              | cbi 0xa,2                                                           | 2                               |
| PORTD  = (1 << PORTD2);            | sbi 0xb,2                                                           | 2                               |
| if (PIND & (1 << PIND2))           | sbis 0x9,2; rjmp .L2                                                | 1 + 2 (presionado) / 2 (suelto) |
| PORTB &= ~(1 << PORTB5);           | cbi 0x5,5; rjmp .L3                                                 | 2 + 2                           |
| PORTB  = (1 << PORTB5);            | sbi 0x5,5                                                           | 2                               |
| PORTB ^= (1 << PORTB0);            | in r24,0x5; eor r24,r25; out 0x5,r24 (con ldi r25,1 antes del lazo) | 3                               |

**Resultado y justificación.** Con el salto de regreso rjmp .L4 (2 ciclos), una vuelta toma  $2+2+2+3+2 = 11$  ciclos  $\approx 0,69 \mu s$  con el pulsador suelto y  $1+2+2+3+2 = 10$  ciclos  $\approx 0,625 \mu s$  presionado. PB0 cambia una vez por vuelta:  $f_{PB0} \approx 1/(2 \cdot 11 \cdot 62,5 \text{ ns}) \approx 727 \text{ kHz}$  (suelto) y  $800 \text{ kHz}$  (presionado), valores calculados, no medidos. Un solo bit usa sbi, cbi y sbis porque esos registros están en 0x00–0x1F; las dos máscaras



de DDRB se juntan en 33 (3 ciclos frente a 4 de dos sbi). ^= usa in/eor/out porque ninguna instrucción de E/S invierte un bit con XOR (EOR no admite constante): el compilador lee, opera y escribe, y no emite el truco de escribir en PINB; el 1 se carga una vez en r25.

## 6. Resultados

Tabla 12. Síntesis de resultados por parte.

| Parte | Resultado clave                                                                             | Evidencia              |
|-------|---------------------------------------------------------------------------------------------|------------------------|
| A     | D13 = PB5; periodo de 1 s (500 ms + 500 ms)                                                 | Figuras 1 y 2          |
| B     | Mismo parpadeo con XOR, OR/AND y PINB ^=; PORTB = deja 00100000                             | Figuras 3 a 6          |
| C     | diagram.json; Tabla 9 (Tabla 4 de la guía); AND/OR/XOR = 10001000, 11101110, 01100110       | Figura 7, Tablas 6 a 9 |
| D     | Promedio: 3,395 $\mu$ s frente a 0,220 $\mu$ s; digitalWrite/registro $\approx$ 15          | Figura 8, Tabla 10     |
| E     | Tabla 11 (Tabla 5 de la guía): CBI, SBI, SBIS, IN, ORI, EOR, OUT; 11 y 10 ciclos por vuelta | Figura 9, Tabla 11     |

## 7. Preguntas de Control

### 7.1. ¿Qué diferencia a un microcontrolador de un microprocesador? Enumere los bloques del ATmega328P que identificó en la práctica.

El microcontrolador ya integra la memoria y los periféricos que el microprocesador necesita fuera. Bloques vistos: CPU AVR de 8 bits, Flash, SRAM, EEPROM (no usada), puertos B y D, Timer0 (delay()) y USART (Monitor Serie).

### 7.2. ¿Qué implica que el ATmega328P tenga arquitectura Harvard? Relacione su respuesta con el uso de la macro F(...) en el Código 3.

Programa y datos están en espacios de direcciones separados: un puntero normal direcciona la SRAM y la Flash solo se lee con LPM. Un texto normal se copia a la SRAM (2 KB) al arrancar; F(...) lo deja solo en Flash (PROGMEM), como en Serial.print(F("Modo: ")).



UNL

Universidad  
Nacional  
de Loja

FEIRNNR - Carrera de Computación

**7.3. Explique por qué  $\text{PORTB} = (1 \ll \text{PORTB5})$ ; puede alterar otras salidas y  $\text{PORTB} |= (1 \ll \text{PORTB5})$ ; no, usando lo observado en los pasos B4 y C6.**

La asignación escribe los ocho bits y deja en 0 todos menos el 5;  $|=$  lee  $\text{PORTB}$ , aplica OR con la máscara y reescribe. En B4 no se notó porque solo se usaba  $\text{PB5}$ ; en C6  $\text{PORTB} = \text{valor}$  borró el bit del LED "L" y este dejó de parpadear.

**7.4. Para usar  $\text{PD2}$  como entrada con pull-up, ¿qué valores deben tener  $\text{DDRD2}$  y  $\text{PORTD2}$ ? ¿Qué se lee en  $\text{PIND2}$  con el pulsador suelto y presionado, y por qué?**

$\text{DDRD2} = 0$  (entrada) y  $\text{PORTD2} = 1$  (activa la pull-up). Suelto se lee 1 porque la pull-up lleva el pin a 5 V; presionado se lee 0 porque el pulsador lo conecta a GND (Tabla 9).

**7.5. Calcule a mano  $0b11001100 \& 0b10101010$ ,  $|$  y  $\wedge$ , y verifique sus resultados con el Monitor Serie.**

Con  $a = 11001100_2$  y  $b = 10101010_2$ :  $\text{AND} = 10001000$ ,  $\text{OR} = 11101110$ ,  $\text{XOR} = 01100110$  (ver C3); coinciden con el Monitor Serie.

**7.6. Si  $a$  es de tipo `uint8_t`, ¿por qué `Serial.println(~a, BIN)` imprime muchos más de 8 bits? ¿Cómo lo corrige el Código 3?**

$a$  se promueve a `int` (16 bits en AVR) antes del  $\sim$ :  $\sim 0x00CC = 0xFF33$ , negativo, que `println` extiende con signo a 32 bits ( $0xFFFFFFFF33$ ). El Código 3 usa `(uint8_t)~a` (queda  $0x33$ ) e `imprimirBin8()`, que imprime exactamente 8 bits.

**7.7. Con los datos de la Parte D, explique por qué `digitalWrite()` es más lento que el acceso directo a registros. ¿Cuándo conviene cada enfoque (portabilidad frente a rendimiento)?**

`digitalWrite()` consulta tablas, comprueba el PWM y protege la escritura con `SREG` y `cli`, mientras el registro es un `sbi/cbi` (Parte D, Tabla 10). La API conviene por portabilidad; los registros, cuando importa la velocidad, a costa de depender del chip.

**7.8. ¿Qué instrucción AVR generó  $\text{PORTB} |= (1 \ll \text{PORTB5})$ ; y cuántos ciclos toma? ¿Por qué  $\text{PORTB} ^= (1 \ll \text{PORTB0})$ ; no se tradujo a una sola instrucción?**

Generó `sbi 0x5,5`, de 2 ciclos.  $\wedge=$  no tiene instrucción de E/S equivalente (`EOR` no admite constante), así que el compilador lee, opera y escribe con `in/eor/out` (3 ciclos, Parte E).



### 7.9. Sostenibilidad: indique dos formas en que lo trabajado contribuye a reducir el consumo energético o los residuos electrónicos.

(a) Simular antes de montar evita comprar y dañar componentes. (b) El acceso por registros hace la misma tarea en  $\approx 15$  veces menos tiempo de CPU (Tabla 10), tiempo que en un sistema real puede pasarse en modo de bajo consumo.

### 7.10. Ventajas y limitaciones de simular en Wokwi frente al hardware real (rebote del pulsador, límites de corriente, tiempos).

Ventajas: no requiere hardware ni daña componentes (Zailani, 2024). Limitaciones: el rebote está desactivado ("bounce": "0"), no se controla el límite de corriente por pin (40 mA absolutos) y los tiempos sirven para comparar, no como valores reales (las tres ejecuciones salieron idénticas).

## 8. Conclusiones

1. El acceso por registros resultó  $\approx 15$  veces más rápido que `digitalWrite()` (0,220  $\mu$ s frente a 3,395  $\mu$ s; orden de magnitud, por el modelo de tiempo del simulador), porque la API consulta tablas, revisa el PWM y guarda SREG.
2. Una vuelta del `while(1)` toma 11 o 10 ciclos: `sbi/cbi` (2 ciclos) para un bit en E/S baja y `in/eor/out` (3 ciclos) para el XOR, lo que da PB0 de  $\approx 727$  y 800 kHz (calculado).
3. Asignar `PORTB =` borra los demás bits (00100000 en B4; LED "L" fijo en C6); `|=`, `&=` y `^=` modifican solo el bit deseado.
4. Escribir un 1 en `PINx` conmuta el pin en hardware en una sola escritura (B3), mientras que `^=` sobre `PORTB` obliga al compilador a leer, operar y escribir (`in/eor/out`, Parte E).

## 9. Recomendaciones

- Tratar el rebote del pulsador en hardware real antes de usar el Código 3.
- Repetir la medición de la Parte D en una placa real para observar su dispersión.
- Preferir `|=`, `&=` y `^=` (o `PINB`) a la asignación directa cuando el puerto tenga varios pines en uso.



unl

Universidad  
Nacional  
de Loja

FEIRNNR - Carrera de Computación

## 10. Declaración de Uso de IA Generativa

Nosotros, **Melissa Contento, Jaime Landazuri, Alejandro Padilla Espinoza, Domenica Rojas e Ivett Zaragocin**, en calidad de autores, declaramos de manera transparente y responsable el uso de herramientas de Inteligencia Artificial Generativa (IAG) durante el desarrollo del presente trabajo, en conformidad con los principios de integridad académica, ética investigativa y buenas prácticas en el uso de tecnologías emergentes.

### 10.1. Alcance del uso de IA generativa

El uso de herramientas de IAG se ha limitado a funciones de apoyo, sin sustituir el juicio crítico ni la autoría intelectual de los autores. En particular, se han utilizado para:

- Comprensión del problema planteado en la guía: algunos integrantes consultaron ChatGPT y Gemini para entender los conceptos de la práctica, sin generar código, simulaciones ni resultados.
- Apoyo en la estructuración, la redacción y el formato del informe.

### 10.2. Clasificación del uso según GAIDeT

De acuerdo con la taxonomía GAIDeT, el uso de IA generativa en este trabajo se clasifica en:

- **Nivel 1 – Asistencia lingüística:** corrección gramatical y mejora de claridad.
- **Nivel 2 – Apoyo en ideación:** consultas para comprender el problema y los conceptos de la práctica.
- **Nivel 3 – Asistencia técnica:** apoyo en la construcción y el formato del documento.

No se ha utilizado IA generativa para realizar las simulaciones, fabricar mediciones ni sustituir la verificación de los resultados de la práctica.

### 10.3. Validación y responsabilidad

Todo contenido asistido por IA ha sido revisado, validado y adaptado por los autores, quienes asumen la responsabilidad total sobre la calidad, veracidad y originalidad del trabajo.

### 10.4. Herramientas utilizadas

- **ChatGPT (OpenAI) y Gemini (Google):** consultados por algunos integrantes únicamente para comprender el problema; no se usaron para programar ni para obtener resultados.
- **Claude (Anthropic):** estructuración, redacción y revisión del informe.

Los códigos, el circuito y las mediciones fueron realizados por los integrantes en Wokwi y Compiler Explorer. Se verificó que los resultados coincidieran con el Monitor Serie, los ciclos con el listado de Compiler Explorer y las características del chip con la hoja de datos.



## Referencias

- Arduino. (2024). *Arduino Language Reference*. Consultado el 3 de octubre de 2026, desde <https://www.arduino.cc/reference/>
- CodeMagic LTD. (2025). *Wokwi – documentación*. Consultado el 3 de octubre de 2026, desde <https://docs.wokwi.com>
- Freire Sánchez, L. A. (2023). *Electrónica y sistemas embebidos: una visión a nivel técnico y tecnológico*. Centro de Investigación y Desarrollo Ecuador. <https://doi.org/10.33996/cide.ecuador.ES2636225>
- Microchip Technology Inc. (2024). *ATmega328P – hoja de datos y documentación del producto*. <https://www.microchip.com/en-us/product/ATmega328P>
- Microchip Technology Inc. (s.f.). *AVR® Instruction Set Manual*. <https://www.microchip.com>
- Valdivieso, C., y Solís, R. (2019). *Microprocesadores: fundamentos y aplicaciones*. LATIn Project. <https://open.umn.edu/opentextbooks/textbooks/microprocesadores-fundamentos-y-aplicaciones>
- Zailani, N. F. (2024). *Getting started with Arduino and ESP32: A beginner's guide with Wokwi*. Politeknik Mersing.

## 11. Anexos

Los listados completos y comentados, copiados del repositorio, siguen a la tabla de enlaces.

Tabla 13. Enlaces públicos del trabajo.

| Parte | Recurso                                                 | Enlace                                                                                                                                    |
|-------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Todas | Repositorio de GitHub (códigos, diagram.json e informe) | <a href="https://github.com/AlejandroTatum/sistemas-digitales-ape1-avr">https://github.com/AlejandroTatum/sistemas-digitales-ape1-avr</a> |
| A     | Código 1                                                | repo: parte-a/                                                                                                                            |
| B     | Wokwi (versión B2)                                      | <a href="https://wokwi.com/projects/476790602516086785">wokwi.com/projects/476790602516086785</a>                                         |
| B     | Código 2, B2, B3 y B4                                   | repo: parte-b/                                                                                                                            |
| C     | Wokwi (Código 3)                                        | <a href="https://wokwi.com/projects/476794088291938305">wokwi.com/projects/476794088291938305</a>                                         |
| C     | Código 3, C6 y diagram.json                             | repo: parte-c/                                                                                                                            |
| D     | Código 4                                                | repo: parte-d/                                                                                                                            |
| E     | Compiler Explorer (AVR gcc 5.4.0, -01 -mmcu=atmega328p) | <a href="https://godbolt.org/z/MYTMrn811">https://godbolt.org/z/MYTMrn811</a>                                                             |
| E     | Código 5                                                | repo: parte-e/                                                                                                                            |

### Código 1. Parpadeo con la API de Arduino (repo: parte-a/codigo1/sketch.ino)

```
1 // APE 1 - Sistemas Digitales (UNL)
2 // Parte A - Código 1
3 // Propósito: línea base que hace parpadear el LED integrado con la API de
  ↳ Arduino
4 // Placa: Arduino UNO (ATmega328P @ 16 MHz), simulación en Wokwi
5 // Pines: D13 = PB5 (LED integrado "L")
6
```



UNL

Universidad  
Nacional  
de Loja

FEIRNNR - Carrera de Computación

```
7 void setup() {
8   pinMode(13, OUTPUT);           // D13 = PB5 (LED integrado "L")
9 }
10
11 void loop() {
12   digitalWrite(13, HIGH);        // enciende el LED
13   delay(500);
14   digitalWrite(13, LOW);         // apaga el LED
15   delay(500);
16 }
```

**Código 2.** Parpadeo por registros con XOR (repo: parte-b/codigo2/sketch.ino)

```
1 // APE 1 - Sistemas Digitales (UNL)
2 // Parte B - Código 2
3 // Propósito: el mismo parpadeo, escribiendo directamente los registros
4 // Placa: Arduino UNO (ATmega328P @ 16 MHz), simulación en Wokwi
5 // Pines: D13 = PB5 (LED integrado "L")
6 // Regla: no se usa pinMode() ni digitalWrite()
7
8 void setup() {
9   DDRB |= (1 << DDB5);           // PB5 (D13) como salida; los demás bits no
    // ↪ cambian
10 }
11
12 void loop() {
13   PORTB ^= (1 << PORTB5);        // XOR: conmuta solo el bit 5
14   delay(500);                    // 500 ms encendido / 500 ms apagado
15 }
```

**Código B2.** Encendido con OR y apagado con AND negado (repo: parte-b/b2/sketch.ino)

```
1 // APE 1 - Sistemas Digitales (UNL)
2 // Parte B - Paso B2
3 // Propósito: encender con OR y apagar con AND negado, con delay(500) entre
    // ↪ ambos
4 // Placa: Arduino UNO (ATmega328P @ 16 MHz), simulación en Wokwi
5 // Pines: D13 = PB5 (LED integrado "L")
6 // Este es el contenido del proyecto de Wokwi de la Parte B.
7
8 void setup() {
9   DDRB |= (1 << DDB5);           // PB5 como salida
10 }
11
12 void loop() {
13   PORTB |= (1 << PORTB5);        // poner a 1 el bit PB5 (enciende el LED)
14   delay(500);
15   PORTB &= ~(1 << PORTB5);      // poner a 0 el bit PB5 (apaga el LED)
16   delay(500);
17 }
```





### Código B3. Conmutación escribiendo en PINB (repo: parte-b/b3/sketch.ino)

```
1 // APE 1 - Sistemas Digitales (UNL)
2 // Parte B - Paso B3
3 // Propósito: conmutar PB5 escribiendo un 1 en PINB
4 // Placa: Arduino UNO (ATmega328P @ 16 MHz), simulación en Wokwi
5 // Pines: D13 = PB5 (LED integrado "L")
6 // En el ATmega328P, escribir 1 en un bit de PINx conmuta el bit de PORTx por
   ↳ hardware.
7
8 void setup() {
9   DDRB |= (1 << DDB5);           // PB5 como salida
10 }
11
12 void loop() {
13   PINB = (1 << PINB5);           // conmuta PB5 sin leer antes PORTB
14   delay(500);
15 }
```

### Código B4. Asignación directa a PORTB (repo: parte-b/b4/sketch.ino)

```
1 // APE 1 - Sistemas Digitales (UNL)
2 // Parte B - Paso B4
3 // Propósito: probar la asignación directa a PORTB en lugar de OR
4 // Placa: Arduino UNO (ATmega328P @ 16 MHz), simulación en Wokwi
5 // Pines: D13 = PB5 (LED integrado "L")
6 // Con la asignación se escribe el byte completo: PB5 queda en 1 y los demás
   ↳ bits en 0.
7 // Aquí funciona porque PB5 es la única salida usada; con más salidas en PORTB
   ↳ las apagaría.
8
9 void setup() {
10   DDRB |= (1 << DDB5);           // PB5 como salida
11 }
12
13 void loop() {
14   PORTB = (1 << PORTB5);         // asignación: enciende PB5 y pone en 0 el
   ↳ resto de PORTB
15   delay(500);
16   PORTB = 0;                     // asignación: apaga PB5 escribiendo el byte
   ↳ completo en 0
17   delay(500);
18 }
```

### Código 3. Laboratorio de bits en los puertos B y D (repo: parte-c/codigo3/sketch.ino)

```
1 /*
2  APE 1 - Sistemas Digitales (UNL)
3  Parte C - Código 3: laboratorio de bits en los puertos B y D
4  Propósito: operaciones a nivel de bit, LEDs en PB0..PB3 con tres modos y
   ↳ pulsador en PD2
5  Placa : Arduino UNO (ATmega328P @ 16 MHz), simulación en Wokwi
6  Libs  : ninguna (avr/io.h llega incluido por Arduino.h)
7  Pines : PB0..PB3 (D8..D11) LEDs | PD2 (D2) pulsador a GND | PB5 (D13) LED L
8  Regla : NO se usa pinMode(), digitalWrite() ni digitalRead()
9  */
```



```
10 const uint8_t MASK_LEDS = 0x0F; // 0b00001111 -> PB0..PB3
11 const uint8_t NUM_MODOS = 3;
12 const uint8_t PATRON_INICIAL[NUM_MODOS] = {0b0000, 0b0001, 0b0101};
13 const unsigned long PERIODO_MS = 300;
14 uint8_t modo = 0;
15 uint8_t patron = PATRON_INICIAL[0];
16 uint8_t btnAnterior = 1; // con pull-up, en reposo se lee 1
17 unsigned long tAnterior = 0;
18 void imprimirBin8(const char *etiqueta, uint8_t v) {
19     Serial.print(etiqueta);
20     for (int8_t i = 7; i >= 0; i--) {
21         Serial.print((v >> i) & 1); // extrae el bit i
22     }
23     Serial.println();
24 }
25 // escribirLeds(): actualiza los LEDs de PB0..PB3
26 void escribirLeds(uint8_t valor) {
27     // leer-modificar-escribir: solo cambian PB0..PB3; PB4..PB7 se conservan
28     PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
29 }
30 void demoOperaciones() {
31     uint8_t a = 0b11001100;
32     uint8_t b = 0b10101010;
33     imprimirBin8("a      = ", a);
34     imprimirBin8("b      = ", b);
35     imprimirBin8("a & b = ", a & b);
36     imprimirBin8("a | b = ", a | b);
37     imprimirBin8("a ^ b = ", a ^ b);
38     imprimirBin8("~a     = ", (uint8_t)~a);
39     imprimirBin8("a << 1 = ", (uint8_t)(a << 1));
40     imprimirBin8("a >> 2 = ", a >> 2);
41     Serial.print("~a sin cast, en BIN: ");
42     Serial.println(~a, BIN); // ver pregunta de control 6
43 }
44 void setup() {
45     DDRB |= MASK_LEDS | (1 << DDB5); // salidas: PB0..PB3 y PB5
46     DDRD &= ~(1 << DDD2); // PD2 entrada (bit DDR = 0)
47     PORTD |= (1 << PORTD2); // pull-up interna (bit PORT = 1)
48     Serial.begin(9600);
49     Serial.println(F("APE 1 - Laboratorio de bits"));
50     demoOperaciones();
51     escribirLeds(patron);
52 }
53 void loop() {
54     // 1) Pulsador: se aísla el bit 2 de PIND (activo en bajo)
55     uint8_t btn = (PIND >> PIND2) & 1; // 1 = suelto, 0 = presionado
56     if (btnAnterior == 1 && btn == 0) { // flanco de bajada
57         modo = (modo + 1) % NUM_MODOS;
58         patron = PATRON_INICIAL[modo];
59         Serial.print(F("Modo: "));
60         Serial.println(modo);
61     }
62     btnAnterior = btn;
63     // 2) Actualización periódica sin bloquear el programa
64     if (millis() - tAnterior >= PERIODO_MS) {
65         tAnterior = millis();
66         switch (modo) {
67             case 0: // contador binario de 4 bits
```



```
68     patron = (patron + 1) & MASK_LEDS;
69     break;
70     case 1: // rotacion circular a la izquierda
71         patron = ((patron << 1) | (patron >> 3)) & MASK_LEDS;
72         break;
73     case 2: // alternancia con XOR
74         patron ^= MASK_LEDS;
75         break;
76     }
77     escribirLeds(patron);
78     PINB = (1 << PINB5); // escribir 1 en PINB5 conmuta PB5
79     imprimirBin8("PORTB = ", PORTB);
80 }
81 }
```

**Código C6.** Único cambio sobre el Código 3: escribirLeds() con asignación directa (repo: parte-c/c6/sketch.ino)

```
1 void escribirLeds(uint8_t valor) {
2     // Paso C6: asignación completa de PORTB en lugar de leer-modificar-escribir
3     PORTB = valor; // asignación directa: sobrescribe también
4     // ↪ PB5 (LED "L")
5 }
```

**Código 4.** Medición de digitalWrite() frente al registro (repo: parte-d/codigo4/sketch.ino)

```
1 // APE 1 - Sistemas Digitales (UNL)
2 // Parte D - Código 4
3 // Propósito: medir el costo de digitalWrite() frente al acceso directo al
4 // ↪ registro
5 // Placa: Arduino UNO (ATmega328P @ 16 MHz), simulación en Wokwi
6 // Pines: D13 = PB5; los resultados salen por el Monitor Serie (9600 baudios)
7
8
9 const uint16_t N = 10000; // repeticiones de cada bucle
10
11 void setup() {
12     Serial.begin(9600);
13     pinMode(13, OUTPUT);
14
15     // Medición 1: API de Arduino
16     unsigned long t0 = micros();
17     for (uint16_t i = 0; i < N; i++) {
18         digitalWrite(13, HIGH);
19         digitalWrite(13, LOW);
20     }
21     unsigned long t1 = micros();
22
23     // Medición 2: acceso directo al registro
24     for (uint16_t i = 0; i < N; i++) {
25         PORTB |= (1 << PORTB5);
26         PORTB &= ~(1 << PORTB5);
27     }
28     unsigned long t2 = micros();
29
30     // Tiempo por escritura = tiempo total / (2 escrituras * N)
31     Serial.print(F("digitalWrite : "));
```



unl

Universidad  
Nacional  
de Loja

FEIRNNR - Carrera de Computación

```
30 Serial.print((t1 - t0) / (2.0 * N), 3);
31 Serial.println(F(" us por escritura"));
32 Serial.print(F("Registro : "));
33 Serial.print((t2 - t1) / (2.0 * N), 3);
34 Serial.println(F(" us por escritura"));
35 }
36
37 void loop() {} // sin trabajo periódico
```

**Código 5.** C puro para AVR analizado en Compiler Explorer (repo: parte-e/codigo5.c)

```
1 /*
2  * APE 1 - Sistemas Digitales (UNL)
3  * Parte E - Código 5: C puro para AVR, sin Arduino
4  * Propósito: analizar el ensamblador generado (SBI, CBI, SBIS, IN, OUT, ORI,
5  *   ↪ EOR)
6  * Dispositivo: ATmega328P | Compilador: avr-gcc 5.4.0, opciones -O1 -mmcu=
7  *   ↪ atmega328p
8  * Pines: PB5 y PB0 salidas | PD2 entrada con pull-up (pulsador a GND)
9  */
10 #include <avr/io.h>
11
12 int main(void) {
13     DDRB |= (1 << DDB5) | (1 << DDB0); // PB5 y PB0 como salidas
14     DDRD &= ~(1 << DDD2); // PD2 como entrada
15     PORTD |= (1 << PORTD2); // pull-up interna en PD2
16
17     while (1) {
18         if (PIND & (1 << PIND2)) { // pulsador suelto (lee 1)
19             PORTB &= ~(1 << PORTB5); // apaga PB5
20         } else {
21             PORTB |= (1 << PORTB5); // enciende PB5
22         }
23         PORTB ^= (1 << PORTB0); // conmuta PB0 en cada vuelta
24     }
25 }
```